

▮ Heap & Priority Queue Mastery

Complete Visual Guide - From Theory to Top K Problems

Master Min Heap, Max Heap & Greedy Algorithms

▮ Table of Contents

1. Introduction - The Pyramid Analogy
2. The Golden Rule (Min vs Max)
3. 7-Step Template for Top K
4. Kth Smallest Element
5. Top K Frequent Elements
6. Merge K Sorted Lists
7. IPO (Capital & Profit)
8. Reorganize String
9. Task Scheduler
10. Summary & Patterns

1 Introduction - The Pyramid Analogy

▮ Heap as a Pyramid

A **Min Heap** is like a human pyramid:

The **lightest person** is at the **top**,
supported by heavier people below.

▮ Visual Structure

Min Heap (Smallest at Top):

```
    1 ← Root (Lightest/Smallest)
  /  \
 3    2 ← Children (Heavier than parent)
/  \
5    4
```

Rule: $\text{Parent} \leq \text{Both Children}$

Max Heap (Largest at Top):

```
   10 ← Root (Heaviest/Largest)
  /  \
 5    8 ← Children (Lighter than parent)
/  \
2    3
```

Rule: $\text{Parent} \geq \text{Both Children}$

⚡ Key Operations

Operation	Java (PriorityQueue)	Time	Description
Peek Top	<code>pq.peak()</code>	$O(1)$	See min/max without removing
Insert	<code>pq.add(x)</code>	$O(\log N)$	Add element, bubble up
Remove Top	<code>pq.poll()</code>	$O(\log N)$	Remove min/max, bubble down
Size	<code>pq.size()</code>	$O(1)$	Get number of elements

▮ When to Use Heap

- ✓ Find **Top K** elements (largest, smallest, frequent)
- ✓ Find **Kth** element (Kth largest, Kth smallest)
- ✓ **Merge K** sorted structures
- ✓ **Greedy** problems (always pick best available)
- ✓ **Scheduling** with priorities
- ✓ **Stream data** where you need min/max

▮ Java Declaration

```
// Min Heap (default - smallest at top)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max Heap (reverse order - largest at top)
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);

// Custom comparator for pairs/objects
PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> a[0] - b[0])
```

▮ Detailed Operations

Insert Operation (Bubble Up)

Insert 2 into Min Heap:

Initial:

```
      3
     / \
    5   4
   / \
  8  10
```

Step 1: Add 2 to bottom (next available spot)

```
      3
     / \
    5   4
   / \ /
  8 10 2 ← New element
```

Step 2: Compare with parent (4)

$2 < 4 \rightarrow$ Swap!

```
      3
     / \
    5   2 ← Swapped
   / \ /
  8 10 4
```

Step 3: Compare with parent (3)

$2 < 3 \rightarrow$ Swap!

```
      2 ← New root!
     / \
    5   3
   / \ /
  8 10 4
```

Bubbled up to correct position!

Time: $O(\log N)$ - height of tree

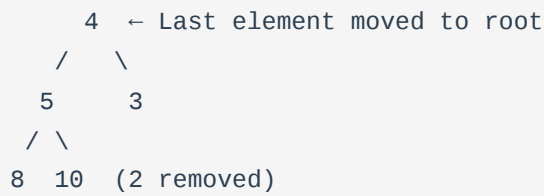
Delete Operation (Bubble Down)

Remove root from Min Heap:

Initial:



Step 1: Swap root with last element



Step 2: Compare with children (5 and 3)

$4 > 3$ (smaller child) → Swap with 3



Step 3: Compare with children (none on right)

4 has no children → STOP

Bubbled down to correct position!

Time: $O(\log N)$

⚠ What Heap CANNOT Do

✗ **Search for specific element:** $O(N)$ - must check all

✗ **Delete specific element:** $O(N)$ - must find it first

✗ **Access middle elements:** Only top is $O(1)$

Heap is ONLY good for:

- ✓ Finding/removing min or max (top element)
- ✓ Maintaining best K elements

▮ Heap on Pairs - Custom Comparators

Problem: Complex Objects in Heap

What if we need to sort by **TWO** properties?

Example: Sort by frequency, then by word (Top K Frequent Words)

▮ Common Patterns

Pattern 1: Min on First, Min on Second

Use Case: Sort pairs by first value, break ties with second

Example: {frequency, word} - both ascending

```
// If first values differ, use first
// If first values same, use second
PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> {
    if (a.first != b.first) {
        return a.first - b.first; // Min on first
    } else {
        return a.second - b.second; // Min on second
    }
});
```

Pattern 2: Min on First, Max on Second

Use Case: Top K Frequent Words (LC 692)

Min heap on frequency, but lexicographically largest word at top

Why? Eliminate least frequent, or alphabetically last word

```
// Top K Frequent Words pattern
class Pair {
    int freq;
    String word;
    Pair(int f, String w) { freq = f; word = w; }
}

// Min Heap on frequency, Max (reverse) on word
PriorityQueue<Pair> pq = new PriorityQueue<>((a, b) -> {
    if (a.freq != b.freq) {
        return a.freq - b.freq; // Min Heap for frequency
    } else {
        // REVERSE for Max Heap behavior on strings
        return b.word.compareTo(a.word); // Lexicographically larger
    }
});

/*
 * Example: If we have:
 * {"the", freq=3}, {"day", freq=3}
 *
 * Compare frequencies: 3 == 3 (same)
 * Compare words: b.word.compareTo(a.word) = "day".compareTo("the")
 * Result: negative (day < the), so "the" comes first
 *
 * This puts "the" at top of heap (lexicographically larger)
 * When we need to eliminate, we remove "the" first
 */
```

Complete Example: Top K Frequent Words

Input: words = ["the", "day", "is", "sunny", "the", "the", "is"], k = 2

Step 1: Count Frequencies

```
{  
  "the": 3,  
  "is": 2,  
  "day": 1,  
  "sunny": 1  
}
```

Step 2: Use Min Heap (size k=2) on frequency

If frequency same, keep lexicographically smaller

Process {"the": 3}:

Heap: [{"the", 3}]

Size: 1 < 2

Process {"is": 2}:

Heap: [{"is", 2}, {"the", 3}]

{"is", 2} ← Root (min freq)

/

{"the", 3}

Size: 2 = 2 → Full!

Process {"day": 1}:

Compare freq 1 with root freq (2)

1 < 2 → "day" has lower frequency → IGNORE

Process {"sunny": 1}:

Same, IGNORE

Result: ["the", "is"]

✓ Key Takeaway

Primary Sort: Determines Min/Max Heap behavior

Secondary Sort: Breaks ties

Reverse Order: Use `b.compareTo(a)` for Max behavior

2 The Golden Rule: Min vs Max Heap

Counter-Intuitive Truth!

To find TOP K LARGEST → Use MIN Heap

To find BOTTOM K SMALLEST → Use MAX Heap

Why This Works - The Elimination Logic

Scenario A: Find Top K Largest

Think of a "**Winners' Bench**" with K seats:

- You want to keep the K **best** players
- The **weakest player** on the bench is the cutoff
- If new player is better than weakest → kick weakest out
- **Min Heap** puts weakest at top for easy removal!

Scenario B: Find Bottom K Smallest

Think of a "**Small Items Bench**" with K seats:

- You want to keep the K **smallest** items
- The **largest** of your small items is the danger

- If new item is smaller than largest → kick largest out
- **Max Heap** puts largest at top for easy removal!

Visual Example: Kth Smallest

Problem: Find 4th Smallest from [10, 5, 4, 3, 48, 2]

Use: MAX Heap of size 4 (keep 4 smallest)

Step 1: Fill bench with first 4 elements

MaxHeap: {10, 5, 4, 3}

```
    10 ← Root (Largest of the 4 smallest)
   /  \
  5    4
 /
3
```

Step 2: New element = 48

Compare 48 with root (10)

48 > 10 → 48 is too large for "smallest" list → IGNORE

Step 3: New element = 2

Compare 2 with root (10)

2 < 10 → 2 deserves a spot!

Action: Remove 10, Insert 2

New MaxHeap: {5, 4, 3, 2}

```
    5 ← New Root (Cutoff)
   /  \
  3    4
 /
2
```

Result: Root (5) is the 4th smallest!

⚠ Common Mistake

DON'T use Max Heap for "largest"!

Max Heap keeps largest at top, but you can't eliminate anything!

Use Min Heap to eliminate the smallest (weakest) from your top K.

3▯ The 7-Step Template for Top K Problems

Universal template that works for **ANY** Top K or Bottom K problem!

▯ The Template Steps

1. **Identify:** Is it a Top/Bottom K problem?
2. **Metric:** What are we comparing? (value, frequency, distance)
3. **Structure:** Create Heap of Pairs {Metric, Element}
4. **Heap Type:** Choose Min (Top K) or Max (Bottom K)
5. **Push K:** Add first K elements unconditionally
6. **Push-Pop:** For remaining elements, add then remove top
7. **Result:** Extract K elements from heap

Template Code

```
/**
 * Universal Top K Template
 * Adjust heap type based on problem
 */
public ResultType topK(InputType[] arr, int k) {
    // Step 4: Choose heap type
    // For TOP K LARGEST → Min Heap
    PriorityQueue<Type> heap = new PriorityQueue<>();

    // Steps 5 & 6: Process all elements
    for (Type element : arr) {
        heap.add(element); // Push

        if (heap.size() > k) {
            heap.poll(); // Pop to maintain size K
        }
    }

    // Step 7: Extract result
    return extractFromHeap(heap);
}
```

4 Problem: Kth Smallest Element

Problem: Find the Kth smallest element in an array

Example: [3, 2, 1, 5, 6, 4], K = 2 → Answer: 2

▮ The Bench Analogy

Imagine a bench with **K seats** for the smallest students:

- The **tallest person** on this bench is the Kth smallest overall
- If someone shorter comes, kick the tallest off the bench
- Use **Max Heap** to track tallest (largest) of the smallest

▮ Complete Walkthrough

Array: [10, 5, 4, 3, 48, 2], Find 3rd Smallest

Iteration 1: Element = 10

MaxHeap: {10}

Size: $1 < 3 \rightarrow$ Keep

Iteration 2: Element = 5

MaxHeap: {10, 5}

Size: $2 < 3 \rightarrow$ Keep

Iteration 3: Element = 4

MaxHeap: {10, 5, 4}

```
    10
   /  \
  5    4
```

Size: $3 = 3 \rightarrow$ Bench full!

Iteration 4: Element = 3

MaxHeap: {10, 5, 4}

Compare 3 with root (10)

$3 < 10 \rightarrow$ Remove 10, Add 3

New MaxHeap: {5, 4, 3}

```
    5
   /  \
  4    3
```

Iteration 5: Element = 48

Compare 48 with root (5)

$48 > 5 \rightarrow$ IGNORE (too large)

Iteration 6: Element = 2

Compare 2 with root (5)

$2 < 5 \rightarrow$ Remove 5, Add 2

Final MaxHeap: {4, 3, 2}

4 ← This is the 3rd smallest!

```
  /  \
 2    3
```

Answer: 4

Java Code (Fully Commented)

```
/**
 * Kth Smallest Element in Array
 *
 * Strategy: Max Heap of size K
 * - Keep K smallest elements
 * - Root is the largest of the smallest = Kth smallest
 *
 * Time: O(N log K)
 * Space: O(K)
 */
public int findKthSmallest(int[] nums, int k) {
    // Max Heap (largest at top)
    PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>((a, b) -> b - a);

    // Process each element
    for (int num : nums) {
        maxHeap.add(num);

        // If heap size exceeds K, remove largest
        // This keeps only the K smallest elements
        if (maxHeap.size() > k) {
            maxHeap.poll(); // Remove largest (root)
        }
    }

    // Root is the largest of the K smallest
    // = Kth smallest overall
}
```

```
    return maxHeap.peak();  
}
```

5 Problem: Top K Frequent Elements

LeetCode 347: Find K most frequent elements

Example: [1,1,1,2,2,3], K = 2 → [1, 2]

Why Min Heap?

Trap: Don't use Max Heap for all N elements!

That's $O(N \log N)$ - same as sorting!

Smart Approach: Use Min Heap of size K

Keep only top K frequencies → $O(N \log K)$

Walkthrough

Input: [1, 1, 1, 2, 2, 3], K = 2

Step 1: Build Frequency Map

{1: 3, 2: 2, 3: 1}

Step 2: Use Min Heap (size K=2)

Process {1: freq=3}:
MinHeap: {1:3}

Size: 1 < 2 → Keep

```
Process {2: freq=2}:
MinHeap: {1:3, 2:2}
    2 ← Min frequency
    /
    1
Size: 2 = 2 → Full!
```

```
Process {3: freq=1}:
MinHeap: {1:3, 2:2}
Compare freq 1 with root freq (2)
1 < 2 → 3 has lower frequency than 2
Action: IGNORE (don't add 3)
```

Result: Heap contains {1, 2} - Top 2 frequent!

Java Code

```
/**
 * Top K Frequent Elements
 * LeetCode 347
 *
 * Time: O(N log K)
 * Space: O(N) for map + O(K) for heap
 */
public int[] topKFrequent(int[] nums, int k) {
    // Step 1: Count frequencies
    Map<Integer, Integer> map = new HashMap<>();
    for (int n : nums) {
        map.put(n, map.getOrDefault(n, 0) + 1);
    }

    // Step 2: Min Heap based on frequency
    // We want TOP K frequencies, so use Min Heap
    PriorityQueue<Integer> minHeap =
        new PriorityQueue<>((a, b) -> map.get(a) - map.get(b));

    // Step 3: Process each unique number
    for (int key : map.keySet()) {
        minHeap.add(key);
    }
}
```



```

        // If heap exceeds K, remove element with min frequency
        if (minHeap.size() > k) {
            minHeap.poll(); // Eliminate weakest
        }
    }

    // Step 4: Extract result
    int[] result = new int[k];
    for (int i = 0; i < k; i++) {
        result[i] = minHeap.poll();
    }
    return result;
}

```

6 Merge K Sorted Lists

Problem: Merge K sorted arrays into one sorted array

Example: $[[1,4,5], [1,3,4], [2,6]] \rightarrow [1,1,2,3,4,4,5,6]$

The Race Tracks Analogy

Imagine **K race tracks** (sorted arrays):

- You can only see the **runner at the start line** of each track
- You need to pick the **fastest (smallest)** runner
- After a runner leaves, the **next runner** from that track comes to start

Problem: Comparing K runners linearly = $O(K)$ per pick

Solution: Use Min Heap = $O(\log K)$ per pick!

Complete Visualization

Three Race Tracks (Arrays):

Track 0: [1, 4, 5]

Track 1: [1, 3, 4]

Track 2: [2, 6]

Step 1: Initialize - First runner from each track

Heap: [{val:1, track:0, pos:0},
 {val:1, track:1, pos:0},
 {val:2, track:2, pos:0}]

Min Heap (by value):

```
      {1,0,0} ← Track 0's runner (1)
      /   \
{1,1,0}   {2,2,0}
```

Step 2: Pick winner (smallest)

Winner: {1, track:0, pos:0}

Add 1 to result

Next from Track 0: {4, track:0, pos:1}

New Heap:

```
      {1,1,0} ← Track 1's runner (1)
      /   \
{2,2,0}   {4,0,1}
```

Step 3: Pick winner

Winner: {1, track:1, pos:0}

Add 1 to result

Next from Track 1: {3, track:1, pos:1}

New Heap:

```
      {2,2,0} ← Track 2's runner (2)
      /   \
{3,1,1}   {4,0,1}
```

Continue until all tracks empty...

Result: [1, 1, 2, 3, 4, 4, 5, 6]

Why This Works

Naive Approach: Check all K pointers = $O(K \times N)$ total

Heap Approach: $O(\log K)$ to find min $\times N$ elements = **$O(N \log K)$**

With $K=100$ lists and $N=10,000$ total elements:

- Naive: 1,000,000 operations
- Heap: 66,000 operations ✓

▮ Complete Java Code

```
/**
 * Merge K Sorted Arrays
 *
 * Time:  $O(N \log K)$  where  $N$  = total elements
 * Space:  $O(K)$  for heap
 */
class Node {
    int value; // The runner's number
    int row;   // Which track (array)
    int col;   // Position in that track

    Node(int v, int r, int c) {
        value = v; row = r; col = c;
    }
}

public List<Integer> mergeKSortedArrays(int[][] arrays) {
    // Min Heap based on value
    PriorityQueue<Node> pq =
        new PriorityQueue<>((a, b) -> a.value - b.value);

    List<Integer> result = new ArrayList<>();

    // Step 1: Add first element from each array
    for (int i = 0; i < arrays.length; i++) {
        if (arrays[i].length > 0) {
            pq.add(new Node(arrays[i][0], i, 0));
        }
    }
}
```

```

    }
}

// Step 2: Process until heap is empty
while (!pq.isEmpty()) {
    Node curr = pq.poll(); // Get smallest (winner)
    result.add(curr.value);

    // Step 3: Add next element from SAME array
    // This is KEY - we track which array it came from!
    if (curr.col + 1 < arrays[curr.row].length) {
        pq.add(new Node(
            arrays[curr.row][curr.col + 1],
            curr.row,
            curr.col + 1
        ));
    }
}

return result;
}

/*
 * Why we need {value, row, col}:
 *
 * We can't just store values in heap because:
 * - When we poll a value, we need to know WHICH array it came from
 * - So we can fetch the NEXT element from that specific array
 *
 * Example: Poll value 1 from array 0
 * Without tracking row: We don't know if it's from array 0 or 1!
 * With tracking: We know it's from array 0, fetch arrays[0][1] next
 */

```

7 IPO Problem (LC 502)

Problem: Maximize capital by selecting up to K projects

Constraint: Can only start project if capital $\geq$ required

The Unlocked Room Analogy

Imagine **two rooms**:

Room 1 (Sorted Array): All projects, locked by cost

- Sorted by capital requirement (cheapest first)
- Projects "unlock" as you gain more money

Room 2 (Max Heap): Projects you can afford NOW

- Organized by profit (highest first)
- Pick the most profitable available project (Greedy)

Complete Walkthrough

```
Projects: [(capital, profit)]
[(0, 1), (1, 2), (1, 3), (2, 5)]
K = 2 (can do 2 projects)
W = 0 (initial capital)
```

```
Step 1: Sort by capital (already sorted)
Room 1 (Locked): [(0,1), (1,2), (1,3), (2,5)]
```

```
Iteration 1:
Current W = 0
```

```
Room 1: Unlock affordable projects
• (0, 1): cost=0 <= W=0? YES → Move to Room 2
```

- (1, 2): cost=1 <= W=0? NO → STOP

Room 2 (Max Heap by profit): [1]

Pick Best: profit=1

New W = 0 + 1 = 1

Iteration 2:

Current W = 1

Room 1: Continue unlocking

- (1, 2): cost=1 <= W=1? YES → Move to Room 2
- (1, 3): cost=1 <= W=1? YES → Move to Room 2
- (2, 5): cost=2 <= W=1? NO → STOP

Room 2 (Max Heap): [3, 2]

```

    3 ← Max profit
  /
 2

```

Pick Best: profit=3

New W = 1 + 3 = 4

Done (K=2 projects completed)

Final Capital: 4

Note: Project (2,5) is now affordable (W=4 >= 2)
but we already did K=2 projects!

□ Why Sort + Heap?

Can't just use Heap for everything!

✗ Heap by Profit only: Might pick expensive project we can't afford

✗ Heap by Capital only: Might pick cheap project with low profit

✓ Sort + Heap solution:

- Sort lets us efficiently find "unlocked" projects as W grows
- Heap lets us greedily pick best profit among affordable
- Pointer ensures we never re-check old projects

Complete Java Code

```
/**
 * IPO - Maximize Capital
 * LeetCode 502
 *
 * Time:  $O(N \log N)$  for sort +  $O(K \log N)$  for heap ops
 * Space:  $O(N)$ 
 */
public int findMaximizedCapital(int k, int w,
                                int[] profits, int[] capital) {
    int n = profits.length;

    // Step 1: Create array of {capital, profit} pairs
    int[][] projects = new int[n][2];
    for (int i = 0; i < n; i++) {
        projects[i][0] = capital[i];
        projects[i][1] = profits[i];
    }

    // Step 2: Sort by capital (Room 1 - locked by cost)
    Arrays.sort(projects, (a, b) -> a[0] - b[0]);

    // Step 3: Max Heap for profits (Room 2 - affordable)
    PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>((a, b) -> b - a);

    int i = 0; // Pointer for sorted array

    // Step 4: Do K projects (or until stuck)
    while (k-- > 0) {
        // THE UNLOCK PHASE:
        // Move ALL affordable projects from Room 1 to Room 2
        while (i < n && projects[i][0] <= w) {
            maxHeap.add(projects[i][1]); // Add profit to heap
            i++; // Move pointer (never go back)
        }

        // THE GREEDY CHOICE:
        // If no projects affordable, we're stuck
    }
}
```

```

        if (maxHeap.isEmpty()) break;

        // Pick project with maximum profit
        w += maxHeap.poll();
    }

    return w;
}

/*
 * Critical Understanding:
 *
 * Q: Why pointer i instead of re-checking all projects?
 * A: Array is sorted! Once a project is unaffordable,
 *    all remaining are also unaffordable.
 *    When W increases, we just continue from i.
 *
 * Q: Why inner while, not if?
 * A: W might have increased a LOT from last project.
 *    Multiple projects might now be affordable.
 *    We need to unlock ALL of them before choosing.
 *
 * Example: W=0 initially, only project (0,1) affordable
 *          After: W=1, projects (1,2) and (1,3) both affordable
 *          Must add BOTH to heap, then pick best (3)
 */

```

8 Reorganize String (LC 767)

Problem: Rearrange string so no two adjacent are same

Example: "aab" → "aba" or "baa"

If impossible: return ""

📌 The PT Teacher / Corner Analogy

Imagine seating students:

- Students line up by **frequency** (Max Heap)
- Always seat the **most frequent** student next (Greedy)
- **Can't seat same student twice in a row!**
- After seating student A, send them to **"the corner"**
- Seat next student B, then bring A back from corner
- The "prev" variable is the corner

📌 Complete Visualization

String: "aaabbc", frequencies: {a:3, b:2, c:1}

Initial State:

MaxHeap (by frequency): {a:3, b:2, c:1}

a:3 ← Most frequent

/ \

b:2 c:1

Corner (prev): null

Result: []

Step 1: Seat 'a'

Pop a from heap

Result: [a]

a still has 2 left → Send to corner

Corner (prev): {a:2}

Heap: {b:2, c:1}

Step 2: Seat 'b'

Pop b from heap

Result: [a, b]

b still has 1 left → Will go to corner

IMPORTANT: Before sending b to corner, bring a back!

Check corner: a exists and has count > 0

Add a back to heap

Heap: {a:2, c:1}

Now send b to corner

Corner (prev): {b:1}

Step 3: Seat 'a' (again)

Pop a from heap

Result: [a, b, a]

a still has 1 left

Bring b back from corner

Heap: {b:1, c:1}

Corner (prev): {a:1}

Step 4: Seat 'b'

Pop b from heap

Result: [a, b, a, b]

b has 0 left (done)

Bring a back from corner

Heap: {a:1, c:1}

Corner (prev): null (b is done)

Step 5: Seat 'a'

Pop a from heap

Result: [a, b, a, b, a]

a has 0 left

Corner (prev): null

Heap: {c:1}

Step 6: Seat 'c'

Pop c from heap

Result: [a, b, a, b, a, c]

Final: "ababac" ✓ (no adjacent duplicates!)

□ Why This Works

Why always pick most frequent?

If you save the most common character for last,
you won't have enough other characters to separate them!

Example: "aaab"

✗ Wrong: Save 'a' for later → "baaa" (adjacent a's!)

✓ Correct: Use 'a' early → "abaa" → "abaa"... wait still fails!

Actually "aaab" is impossible - (a=3, others=1, total=4)

Need at least 3 others to separate 3 a's!

Impossibility condition:

If any character appears more than $(n+1)/2$ times → impossible

Complete Java Code

```
/**
 * Reorganize String
 * LeetCode 767
 *
 * Time:  $O(N \log 26) = O(N)$ 
 * Space:  $O(26) = O(1)$ 
 */
public String reorganizeString(String s) {
    // Step 1: Count frequencies
    int[] counts = new int[26];
    for (char c : s.toCharArray()) {
        counts[c - 'a']++;
    }

    // Step 2: Max Heap by frequency
    // Store character indices (0-25), sorted by their counts
    PriorityQueue<Integer> pq =
        new PriorityQueue<>((a, b) -> counts[b] - counts[a]);

    for (int i = 0; i < 26; i++) {
        if (counts[i] > 0) {
            pq.add(i);
        }
    }

    StringBuilder sb = new StringBuilder();
    int prev = -1; // The "corner" - holds previous character
```

```

// Step 3: Greedy placement
while (!pq.isEmpty()) {
    int curr = pq.poll(); // Pick most frequent

    // Add current character to result
    sb.append((char) (curr + 'a'));
    counts[curr]--;

    // CRITICAL: Bring previous character back from corner
    // We do this BEFORE sending current to corner
    if (prev >= 0 && counts[prev] > 0) {
        pq.add(prev); // Previous is now valid (non-adjacent)
    }

    // Send current to the corner
    // It can't be used in next position (would be adjacent)
    prev = curr;
}

// Step 4: Check if we used all characters
if (sb.length() != s.length()) {
    return ""; // Impossible
}

return sb.toString();
}

```

```

/*
 * Critical Understanding:
 *
 * Q: Why update prev AFTER adding prev back to heap?
 * A: The sequence is:
 *
 * 1. Use current character
 * 2. Bring OLD prev back (it's now far enough)
 * 3. Make current the NEW prev (can't use it next)
 *
 * Q: What if we did it the other way?
 * A: If we set prev = curr first, then add prev to heap,
 *    we'd be adding the SAME character we just used!
 *
 * Q: Why check sb.length() != s.length()?

```

```

* A: If a character is too frequent, we'll run out of
*   other characters to place between them.
*   The heap will empty early, leaving characters unused.
*
* Example impossible case: "aaab"
* After: "a b a a" - only 4 positions, but a in prev with count=1
* Heap empty but prev still has a character → Failed!
* Result length = 3, but input length = 4 → return ""
*/

```

9 Task Scheduler (LC 621)

Problem: Schedule tasks with n-interval cooling period

Example: tasks = [A,A,A,B,B,B], n = 2 → Output: 8

The PT Teacher Analogy

Imagine a PE class:

- Students (Tasks) line up by how many laps they need to run
- Most laps goes first (Greedy - use Max Heap)
- After running, student must **rest for n intervals**
- Resting students sit in a "Waiting Room" (Queue)
- When rest time is over, they rejoin the line
- If no student ready, teacher waits (IDLE)

Complete Walkthrough

Tasks: [A, A, A, B, B, B], n = 2 (gap between same tasks)

Frequencies: {A: 3, B: 3}

Time = 0:

MaxHeap: {A:3, B:3}

WaitQueue: []

Time = 1: Execute A

Action: Pop A from heap

Execute: A

Remaining: A has 2 left

Send to waiting room: {A:2, returnTime: 1+2+1=4}

MaxHeap: {B:3}

WaitQueue: [{A:2, time:4}]

CPU: [A]

Time = 2: Execute B

Action: Pop B from heap

Execute: B

Send to waiting: {B:2, returnTime:5}

MaxHeap: {}

WaitQueue: [{A:2, t:4}, {B:2, t:5}]

CPU: [A, B]

Time = 3: IDLE

MaxHeap is empty!

Check WaitQueue: A returns at time 4 (not yet)

Action: CPU IDLE

MaxHeap: {}

WaitQueue: [{A:2, t:4}, {B:2, t:5}]

CPU: [A, B, idle]

Time = 4: Execute A

Check WaitQueue: A ready! (returnTime = 4)

Move A to MaxHeap

Pop A from heap

Execute: A

Send to waiting: {A:1, returnTime:7}

MaxHeap: {}

WaitQueue: [{B:2, t:5}, {A:1, t:7}]

CPU: [A, B, idle, A]

Time = 5: Execute B

Check WaitQueue: B ready!

Move B to MaxHeap

Execute: B

Send to waiting: {B:1, returnTime:8}

CPU: [A, B, idle, A, B]

Time = 6: IDLE

CPU: [A, B, idle, A, B, idle]

Time = 7: Execute A (last A)

CPU: [A, B, idle, A, B, idle, A]

Time = 8: Execute B (last B)

CPU: [A, B, idle, A, B, idle, A, B]

Total Time: 8

□ Key Decisions

Why Max Heap?

Always execute most frequent task first (Greedy)

Ensures we spread out common tasks

Why Queue?

Tasks cooling down can't be executed yet

Need to track when each task can return

Why Simulation?

Must track time step-by-step

CPU might be idle if all tasks cooling

□ Complete Java Code

```
/**
 * Task Scheduler (CPU Scheduling)
 * LeetCode 621
 *
 * Time:  $O(N)$  where  $N$  = number of tasks
 * Space:  $O(26) = O(1)$  for frequencies
```

```

*/
public int leastInterval(char[] tasks, int n) {
    // Step 1: Count frequencies
    Map<Character, Integer> counts = new HashMap<>();
    for (char c : tasks) {
        counts.put(c, counts.getOrDefault(c, 0) + 1);
    }

    // Step 2: Max Heap (most frequent first)
    PriorityQueue<Integer> maxHeap =
        new PriorityQueue<>((a, b) -> b - a);
    maxHeap.addAll(counts.values());

    // Step 3: Waiting room (Queue)
    // Stores {frequency, returnTime}
    Queue<int[]> waitList = new LinkedList<>();

    int time = 0;

    // Step 4: Simulate time step by step
    while (!maxHeap.isEmpty() || !waitList.isEmpty()) {
        time++; // Advance time

        // If heap has tasks, execute one
        if (!maxHeap.isEmpty()) {
            int freq = maxHeap.poll(); // Execute task

            // If task has more executions left
            if (freq - 1 > 0) {
                // Send to waiting room with return time
                waitList.add(new int[]{freq - 1, time + n});
            }
        }

        // Else: CPU is IDLE (time still advances)

        // Check if any task's cooling period is over
        if (!waitList.isEmpty() && waitList.peek()[1] == time) {
            // Move back to heap
            maxHeap.add(waitList.poll()[0]);
        }
    }
}

```



```

    return time;
}

/*
 * Critical Understanding:
 *
 * Q: Why not just calculate mathematically?
 * A: Because of idle time complexity.
 *     Formula works for simple cases, but simulation handles all cases.
 *
 * Q: Why check waitList AFTER executing?
 * A: A task executed at time T can't return until T+n.
 *     We check if tasks from PREVIOUS cycles are ready now.
 *
 * Q: What if heap empty but waitList not?
 * A: CPU IDLE - time advances but no task executes.
 *     This is why we check "!maxHeap.isEmpty() || !waitList.isEmpty()"
 */

```

□ Summary & Patterns

□ Pattern Recognition Table

Problem Type	Heap Type	Key Insight
Top K Largest	Min Heap (size K)	Eliminate smallest from top K
Bottom K Smallest	Max Heap (size K)	Eliminate largest from bottom K
Kth Largest	Min Heap (size K)	Root is Kth largest

Kth Smallest	Max Heap (size K)	Root is Kth smallest
Merge K Sorted	Min Heap	Track {value, row, col}
Greedy Choice	Max Heap	Always pick best available
With Cooldown	Max Heap + Queue	Hold prev, use wait list
Two Properties	Sort + Heap	Sort one, heap other

Decision Flowchart

Question: Find Top K?

- └ Top K LARGEST → Use MIN Heap
- └ Bottom K SMALLEST → Use MAX Heap

Question: Greedy choice?

- └ Always pick BEST → Use MAX Heap

Question: Merge K structures?

- └ Use MIN Heap with tuples

Question: Cooldown/Gap?

- └ Use MAX Heap + Wait Queue

Question: Two constraints?

- └ SORT one, HEAP the other

⚡ Complexity Cheat Sheet

Top K from N elements:

- Brute Force (Sort all): $O(N \log N)$

- Heap Approach: $O(N \log K)$ ✓ Better!

Merge K lists (total N):

- $O(N \log K)$ - much better than $O(K \times N)$!

Space:

- Heap size: $O(K)$ or $O(N)$
- Always better than $O(N^2)$ approaches

🏆 You've Mastered Heaps!

You now understand:

- The Golden Rule (Min for Top, Max for Bottom)
 - 7-Step Template for Top K
 - Elimination logic
 - Greedy + Heap patterns
 - All major heap problems!

Practice these to become a Heap master!